

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий

Лабораторная работа №1

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Реализация многослойного персептрона для решения задачи «исключающее ИЛИ»»

Выполнил:

Студент 4 курса

Факультета ЭИС

Группы ИИ-26

Сугак В.А.

Проверил:

Никитин С.К.

Брест 2026

Цель работы: реализовать многослойный персептрон (MLP) архитектуры 2-2-1 с сигмоидной функцией активации для решения задачи типа «исключающее ИЛИ» (XOR), в которой входные и выходные значения представлены не бинарными 0 и 1, а произвольными вещественными (целыми) числами из диапазона от -10 до 10 , соответствующими логическим классам 0 и 1. Провести сравнение эффективности обучения и качества решения задачи многослойным персептроном в сопоставлении с однослойным персептроном, показав принципиальную линейную неразделимость задачи XOR.

Задачи лабораторной работы

1. Реализовать многослойный персептрон архитектуры 2-2-1 (2 входных нейрона, 2 нейрона скрытого слоя, 1 выходной нейрон) с сигмоидной функцией активации на всех слоях.
2. Реализовать алгоритм обучения сети методом обратного распространения ошибки (backpropagation) в онлайн-режиме (обновление весов после каждого примера).
3. Обучить сеть на полной таблице истинности (4 примера) своего варианта до достижения заданного критерия остановки по суммарной ошибке $E_s \leq E_e$.
4. Реализовать и обучить однослойный персептрон (без скрытого слоя, один сигмоидный нейрон) на тех же данных и зафиксировать результат его обучения.
5. Сравнить многослойный и однослойный персептроны по следующим критериям:
 - количество эпох до сходимости (либо факт недостижения критерия остановки за заданное максимальное число эпох),
 - итоговая суммарная ошибка, точность (ассигасу) воспроизведения всех 4 примеров таблицы истинности.
6. Реализовать режим функционирования обученной сети: пользователь вводит пару чисел (A, B) из диапазона $[-10; 10]$ (не обязательно совпадающих с c_0 и c_1), сеть выдаёт выходное значение $\hat{y}$ и указывает, к какому классу (c_0 или c_1) оно ближе.
7. Сформулировать вывод по результатам сравнения (см. вопросы ниже)

Вариант 4 ($c_0 = 7$, $c_1 = -7$)

A	B	$A \oplus B$
7	7	7
7	-7	-7
-7	7	-7
-7	-7	7

Ход работы

Архитектура: MLP 2-2-1 - два входных нейрона (A, B), два нейрона скрытого слоя, один выходной нейрон. Активация - сигмоида на всех слоях. SLP (однослойный персептрон) - два входа напрямую соединены с одним выходным нейроном, активация тоже сигмоида, скрытого слоя нет.

Результаты обучения:

MLP: сходимость достигнута на 5161 эпохе, итоговая ошибка $E_s \approx 0.000947$. Accuracy 100% - сеть правильно воспроизводит логику XOR по всей таблице: для одинаковых по знаку входов (7, 7) и (-7, -7) выход близок к 7 (6.598 и 7.014), а для разных по знаку входов (7, -7) и (-7, 7) выход близок к -7 (-6.710 и -6.774).

SLP: сходимость не достигнута - за все 100000 эпох ошибка осталась высокой, $E_s \approx 0.752186$. Ассигасу всего 50% - сеть не смогла выучить XOR: выходы для (7, 7) и (7, -7) почти одинаковые (-2.941 и -2.920), хотя должны сильно отличаться. Это показывает, что линейная модель не может разделить классы XOR.

Код программы:

```
import numpy as np

np.random.seed(42)

C0 = 7
C1 = -7

X_raw = np.array([[C0, C0], [C0, C1], [C1, C0], [C1, C1]], dtype=float)
Y_raw = np.array([[C0], [C1], [C1], [C0]], dtype=float)

RANGE_MIN, RANGE_MAX = -10, 10

def normalize(v):
    return (v - RANGE_MIN) / (RANGE_MAX - RANGE_MIN)

def denormalize(v):
    return v * (RANGE_MAX - RANGE_MIN) + RANGE_MIN

X = normalize(X_raw)
Y = normalize(Y_raw)

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-x))

def sigmoid_deriv(out):
    return out * (1.0 - out)

def closest_class(y):
    return C0 if abs(y - C0) <= abs(y - C1) else C1

class MLP:
    def __init__(self, lr=5.0):
        self.lr = lr
        self.W1 = np.random.uniform(-0.5, 0.5, (2, 2))
        self.b1 = np.random.uniform(-0.5, 0.5, (1, 2))
        self.W2 = np.random.uniform(-0.5, 0.5, (2, 1))
        self.b2 = np.random.uniform(-0.5, 0.5, (1, 1))

    def forward(self, x):
        x = x.reshape(1, -1)
        self.x = x
        self.a1 = sigmoid(x @ self.W1 + self.b1)
        self.a2 = sigmoid(self.a1 @ self.W2 + self.b2)
        return self.a2

    def backward(self, target):
        target = target.reshape(1, -1)
        error_out = target - self.a2
        delta_out = error_out * sigmoid_deriv(self.a2)
        delta_hid = (delta_out @ self.W2.T) * sigmoid_deriv(self.a1)

        self.W2 += self.lr * (self.a1.T @ delta_out)
        self.b2 += self.lr * delta_out
        self.W1 += self.lr * (self.x.T @ delta_hid)
        self.b1 += self.lr * delta_hid

        return np.sum(error_out ** 2)

    def train(self, X, Y, Ee=0.001, max_epochs=100000):
        for epoch in range(1, max_epochs + 1):
            Es = 0.0
            for i in np.random.permutation(len(X)):
                self.forward(X[i])
                Es += self.backward(Y[i])
            if Es <= Ee:
                return epoch, Es
```

```

        return max_epochs, Es

def predict(self, x):
    return self.forward(np.array(x)).flatten()[0]

class SingleLayerPerceptron:
    def __init__(self, lr=5.0):
        self.lr = lr
        self.W = np.random.uniform(-0.5, 0.5, (2, 1))
        self.b = np.random.uniform(-0.5, 0.5, (1, 1))

    def forward(self, x):
        x = x.reshape(1, -1)
        self.x = x
        self.a = sigmoid(x @ self.W + self.b)
        return self.a

    def backward(self, target):
        target = target.reshape(1, -1)
        error = target - self.a
        delta = error * sigmoid_deriv(self.a)
        self.W += self.lr * (self.x.T @ delta)
        self.b += self.lr * delta
        return np.sum(error ** 2)

    def train(self, X, Y, Ee=0.001, max_epochs=100000):
        for epoch in range(1, max_epochs + 1):
            Es = 0.0
            for i in np.random.permutation(len(X)):
                self.forward(X[i])
                Es += self.backward(Y[i])
            if Es <= Ee:
                return epoch, Es
        return max_epochs, Es

    def predict(self, x):
        return self.forward(np.array(x)).flatten()[0]

def evaluate(model, name):
    print(f"\n{name}")
    correct = 0
    for i in range(len(X)):
        y_pred = denormalize(model.predict(X[i]))
        pred_class = closest_class(y_pred)
        target_class = Y_raw[i][0]
        ok = pred_class == target_class
        correct += ok
        print(f"A={X_raw[i][0]:>3.0f} B={X_raw[i][1]:>3.0f} | "
              f"y_hat={y_pred:>7.3f} | класс {pred_class:>3.0f} | "
              f"ожд {target_class:>3.0f} | {'OK' if ok else 'WRONG'}")
    return correct / len(X)

def run_network(model, A, B):
    y = denormalize(model.predict(normalize(np.array([A, B], dtype=float))))
    return y, closest_class(y)

Ee = 0.001
MAX_EPOCHS = 100000

mlp = MLP()
mlp_epochs, mlp_Es = mlp.train(X, Y, Ee, MAX_EPOCHS)
mlp_acc = evaluate(mlp, "MLP 2-2-1")

slp = SingleLayerPerceptron()
slp_epochs, slp_Es = slp.train(X, Y, Ee, MAX_EPOCHS)

```

```

slp_acc = evaluate(slp, "Однослойный персептрон")

print("\nСРАВНЕНИЕ")
print(f"{'Модель':<25}{'Эпох':<10}{'Es':<12}{'Accuracy'}")
print(f"{'MLP 2-2-1':<25}{mlp_epochs:<10}{mlp_Es:<12.6f}{mlp_acc*100:.0f}%")
print(f"{'Однослойный':<25}{slp_epochs:<10}{slp_Es:<12.6f}{slp_acc*100:.0f}%")

print("\nРЕЖИМ ФУНКЦИОНИРОВАНИЯ (MLP)")
for A, B in [(7, 7), (7, -7), (-7, 7), (-7, -7), (8, -6), (0, 0)]:
    y, cls = run_network(mlp, A, B)
    print(f"A={A:>4} B={B:>4} | y_hat={y:>7.3f} | класс {cls}")

print("\nВВОД ПОЛЬЗОВАТЕЛЯ ('q' для выхода)")
while True:
    a_in = input("A: ").strip()
    if a_in.lower() == 'q':
        break
    b_in = input("B: ").strip()
    if b_in.lower() == 'q':
        break
    try:
        y, cls = run_network(mlp, float(a_in), float(b_in))
        print(f"y_hat={y:.3f} | класс {cls}\n")
    except ValueError:
        print("Некорректный ввод\n")

```

Вывод программы:

```

MLP 2-2-1
A=  7 B=  7 | y_hat=  6.598 | класс  7 | ожид  7 | OK
A=  7 B= -7 | y_hat= -6.710 | класс -7 | ожид -7 | OK
A= -7 B=  7 | y_hat= -6.774 | класс -7 | ожид -7 | OK
A= -7 B= -7 | y_hat=  7.014 | класс  7 | ожид  7 | OK

Однослойный персептрон
A=  7 B=  7 | y_hat= -2.941 | класс -7 | ожид  7 | WRONG
A=  7 B= -7 | y_hat= -2.920 | класс -7 | ожид -7 | OK
A= -7 B=  7 | y_hat= -2.434 | класс -7 | ожид -7 | OK
A= -7 B= -7 | y_hat= -2.413 | класс -7 | ожид  7 | WRONG

СРАВНЕНИЕ
Модель          Эпох      Es      Accuracy
MLP 2-2-1        5161     0.000947 100%
Однослойный     100000    0.752186  50%

РЕЖИМ ФУНКЦИОНИРОВАНИЯ (MLP)
A=  7 B=  7 | y_hat=  6.598 | класс  7
A=  7 B= -7 | y_hat= -6.710 | класс -7
A= -7 B=  7 | y_hat= -6.774 | класс -7
A= -7 B= -7 | y_hat=  7.014 | класс  7
A=  8 B= -6 | y_hat= -5.160 | класс -7
A=  0 B=  0 | y_hat= -6.742 | класс -7

```

```

ВВОД ПОЛЬЗОВАТЕЛЯ ('q' для выхода)
A: 0
B: 1
y_hat=-6.056 | класс -7

A: -7
B: 7
y_hat=-6.774 | класс -7

A: 10
B: -10
y_hat=-6.696 | класс -7

A: q

```

Однослойный персептрон не смог решить задачу XOR: после 100000 эпох ошибка осталась высокой ($E_s = 0.752$), а точность составила лишь 50%, поскольку он способен провести только одну прямую разделяющую границу, а классы XOR расположены так, что одной прямой их не разделить. Многослойный персептрон 2-2-1, напротив, полностью решил задачу с точностью 100%, достигнув заданного порога ошибки за 5161 эпоху благодаря скрытому слою, который строит несколько промежуточных границ и комбинирует их в нелинейную разделяющую поверхность. Поскольку сигмоида ограничена диапазоном (0, 1), а классы в задаче равны 7 и -7, входные и целевые значения пришлось нормализовать в [0, 1] перед обучением и денормализовать выход обратно в [-10, 10], из-за чего сеть выдаёт не точное значение класса, а близкое к нему число, которое затем сопоставляется

с ближайшим классом. При этом переход от классических значений 0 и 1 к диапазону $[-10, 10]$ не изменил сути задачи: линейная неразделимость XOR зависит только от взаимного расположения точек классов, а не от конкретных чисел, которыми они обозначены, поэтому полученные результаты полностью согласуются с классическим случаем и лишь потребовали дополнительной нормализации данных.

Вывод: реализовала многослойный персептрон (MLP) архитектуры 2-2-1 с сигмоидной функцией активации для решения задачи типа «исключающее ИЛИ» (XOR), в которой входные и выходные значения представлены не бинарными 0 и 1, а произвольными вещественными (целыми) числами из диапазона от -10 до 10, соответствующими логическим классам 0 и 1. Провела сравнение эффективности обучения и качества решения задачи многослойным персептроном в сопоставлении с однослойным персептроном, показала принципиальную линейную неразделимость задачи XOR.